

FC He-Init Hyperparameter Tuning: Results and Analysis

Last updated: 2026-05-04

1. Experiment Design

- **Architecture family:** Plain FC (fully-connected), width 500, ReLU activations, He initialization
- **Comparison axes:** BN vs NoBN × depths {30, 50, 100} × datasets {CIFAR-10, Fashion-MNIST}
- **12 architectures** total (2 BN settings × 3 depths × 2 datasets)
- **Goal:** For each architecture, find hyperparameters that minimize training error, then compare
- **Criterion:** eval_train_accuracy >= 0.995 AND eval_train_loss <= 0.10
- **eval_train:** accuracy and loss computed over the entire training set in evaluation mode (model.eval()), as opposed to train-mode batch averages

2. Hyperparameter Search Space

2.1 Optimizers

- **Adam:** adaptive learning rate per parameter using running estimates of gradient mean and variance
- **SGD with momentum (beta=0.9):** classical stochastic gradient descent. Momentum accumulates a velocity vector — each update is a weighted average of the current gradient and the previous velocity, which smooths the trajectory and helps cross flat regions

2.2 Learning Rates

Setting	Values
NoBN + Adam	3e-4, 1e-3
NoBN + SGD	1e-2, 3e-2
BN + Adam	1e-3, 3e-3
BN + SGD	3e-2, 1e-1

2.3 LR Schedulers

Scheduler	Description
None	Constant LR throughout training
Cosine	LR follows a cosine curve from the initial value down to approx. 0 over the course of training. Starts aggressive, then gradually fine-tunes
OneCycle	A two-phase schedule: (1) warmup — LR gradually increases from a small value (LR/25) up to the peak LR over the first 30% of training, then (2) annealing — LR decays from the peak all the way down to a very small value (peak/10000). The warmup phase is the key idea: it lets the network slowly find a good region of the loss landscape before committing to large updates. Proposed by Smith & Topin (2019)

2.4 Batch Size

- **NoBN**: 128
- **BN**: 128 and 256. BN computes per-batch statistics, so larger batches give more stable mean/variance estimates

2.5 BN Momentum

Controls how fast BN updates its running mean and variance:

$\text{running_mean} = (1 - m) * \text{running_mean} + m * \text{batch_mean}$

- **m = 0.1**: PyTorch default. Each batch contributes 10% to the running statistics
- **m = 0.05**: Slower, more conservative updates

2.6 Fixed Parameters

Parameter	Value
Weight decay	0
Width	500
Bias init	Zero

3. Sweep Grid Summary

BN Setting	Configs per Architecture
NoBN	2 optimizers × {2 LRs} × 3 schedulers × 1 batch size = 12
BN	2 optimizers × {2 LRs} × 2 schedulers × 2 batch sizes × 2 BN momentums = 32

Sweep training: 60 epochs per run, early stopping at $\text{eval_train} \geq 0.995$ for 3 consecutive epochs. **Targeted rerun**: Best config per architecture, 200 epochs, patience=5. All 12 architectures were rerun with their best config.

4. Results: Targeted Best-of-12 (200 epochs)

4.1 Hyperparameter Configurations

#	Dataset	Depth	BN	Optimizer	LR	Scheduler	BS	BN-m
1	CIFAR-10	30L	No	SGD	0.01	OneCycle	128	—
2	CIFAR-10	30L	Yes	Adam	0.001	OneCycle	256	0.1
3	F-MNIST	30L	No	SGD	0.01	OneCycle	128	—
4	F-MNIST	30L	Yes	Adam	0.001	Cosine	256	0.1
5	CIFAR-10	50L	No	SGD	0.01	OneCycle	128	—
6	CIFAR-10	50L	Yes	Adam	0.001	Cosine	256	0.05
7	F-MNIST	50L	No	SGD	0.01	OneCycle	128	—

#	Dataset	Depth	BN	Optimizer	LR	Scheduler	BS	BN-m
8	F-MNIST	50L	Yes	Adam	0.001	OneCycle	256	0.05
9	CIFAR-10	100L	No	Adam	0.001	Cosine	128	—
10	CIFAR-10	100L	Yes	Adam	0.001	Cosine	256	0.05
11	F-MNIST	100L	No	Adam	0.001	Cosine	128	—
12	F-MNIST	100L	Yes	Adam	0.001	OneCycle	256	0.05

4.2 Results: Best vs Final Epoch

The tables below report both **peak** metrics (best across all epochs, with the epoch at which they occurred) and **final** metrics (last epoch of training). For passing architectures, the two are nearly identical. For failing architectures, the gap reveals training collapse — the network learned something and then lost it.

Peak metrics (best value seen at any epoch during training):

#	Architecture	Best eval_train	@ epoch	Best test	@ epoch
1	cifar10/30L/NoBN	0.9974	121	0.5457	99
2	cifar10/30L/BN	0.9976	132	0.5508	122
3	fmnist/30L/NoBN	0.9977	101	0.8978	76
4	fmnist/30L/BN	0.9972	101	0.9015	103
5	cifar10/50L/NoBN	0.9981	137	0.5415	97
6	cifar10/50L/BN	0.5423	38	0.4600	45
7	fmnist/50L/NoBN	0.9291	21	0.8842	21
8	fmnist/50L/BN	0.8868	39	0.8372	39
9	cifar10/100L/NoBN	0.1000	1	0.1000	1
10	cifar10/100L/BN	0.1644	136	0.1681	136
11	fmnist/100L/NoBN	0.1000	1	0.1000	1
12	fmnist/100L/BN	—	—	—	—

Final epoch (where the network stands when training ends):

#	Architecture	Final eval_train	Final eval_loss	Final test	Last epoch	Result
1	cifar10/30L/NoBN	0.9974	0.0086	0.5425	121	PASS
2	cifar10/30L/BN	0.9976	0.0124	0.5486	132	PASS
3	fmnist/30L/NoBN	0.9960	0.0107	0.8957	102	PASS
4	fmnist/30L/BN	0.9963	0.0183	0.9015	103	PASS
5	cifar10/50L/NoBN	0.9981	0.0073	0.5381	137	PASS
6	cifar10/50L/BN	0.2215	2.3510	0.2251	200	FAIL
7	fmnist/50L/NoBN	0.8642	0.4040	0.8414	200	FAIL
8	fmnist/50L/BN	0.2960	2.8676	0.2927	200	FAIL
9	cifar10/100L/NoBN	0.1000	2.3026	0.1000	200	FAIL
10	cifar10/100L/BN	0.1211	2.2937	0.1305	200	FAIL
11	fmnist/100L/NoBN	0.1000	2.3026	0.1000	200	FAIL
12	fmnist/100L/BN	—	—	—	—	Timed out

Score: 5 of 11 completed runs pass the criterion. 1 run (fmnist/100L/BN) timed out before completing.

4.3 Training Collapse in Failing Architectures

The gap between best and final metrics quantifies how much the network *unlearned* during training:

#	Architecture	Δ eval_train (best→final)	Δ test (best→final)	Interpretation
6	cifar10/50L/BN	0.5423 → 0.2215 (-0.32)	0.4600 → 0.2251 (-0.23)	BN running stats degrade; network collapses to near-random
7	fmnist/50L/NoBN	0.9291 → 0.8642 (-0.06)	0.8842 → 0.8414 (-0.04)	Mild degradation; partially trained but couldn't reach criterion
8	fmnist/50L/BN	0.8868 → 0.2960 (-0.59)	0.8372 → 0.2927 (-0.54)	Catastrophic collapse: learned representations destroyed by epoch 200
10	cifar10/100L/BN	0.1644 → 0.1211 (-0.04)	0.1681 → 0.1305 (-0.04)	Flickering near random chance; never truly learned

The BN architectures (runs 6, 8) show the largest collapses. The network achieves reasonable accuracy early on (when BN batch statistics are still close to the running statistics), but as training progresses the running statistics drift, causing eval-mode performance to deteriorate. This is not overfitting — the *training* eval also collapses.

5. Training Curves

5.1 Eval-Train Accuracy Overview

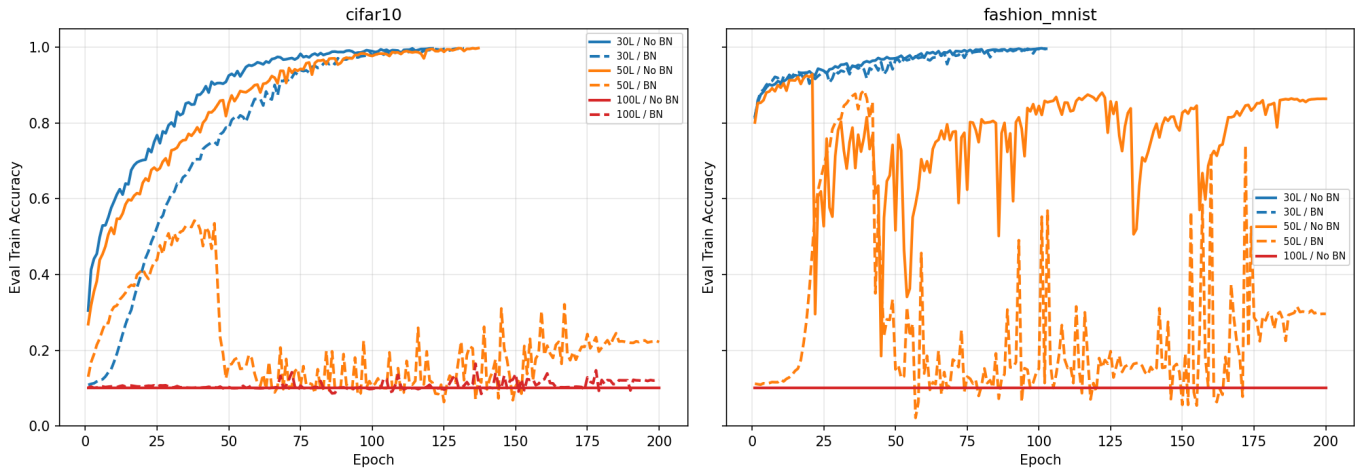


Figure 1: Eval-train accuracy: both datasets

5.2 CIFAR-10 Depth Panels

5.3 Fashion-MNIST Depth Panels

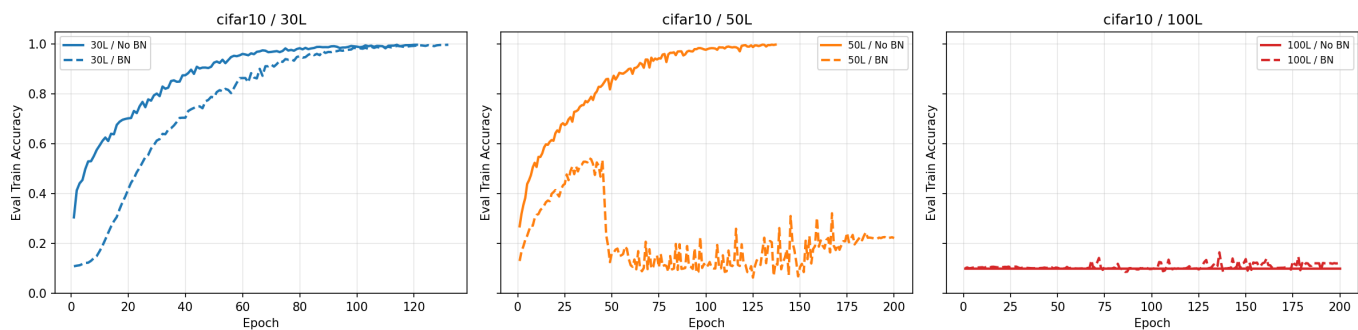


Figure 2: CIFAR-10 eval-train accuracy by depth

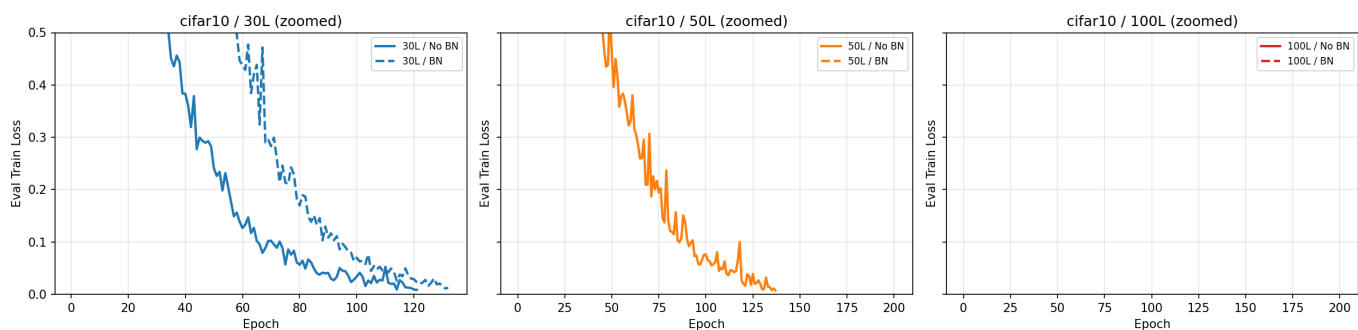


Figure 3: CIFAR-10 eval-train loss by depth (zoomed to 0-0.5)

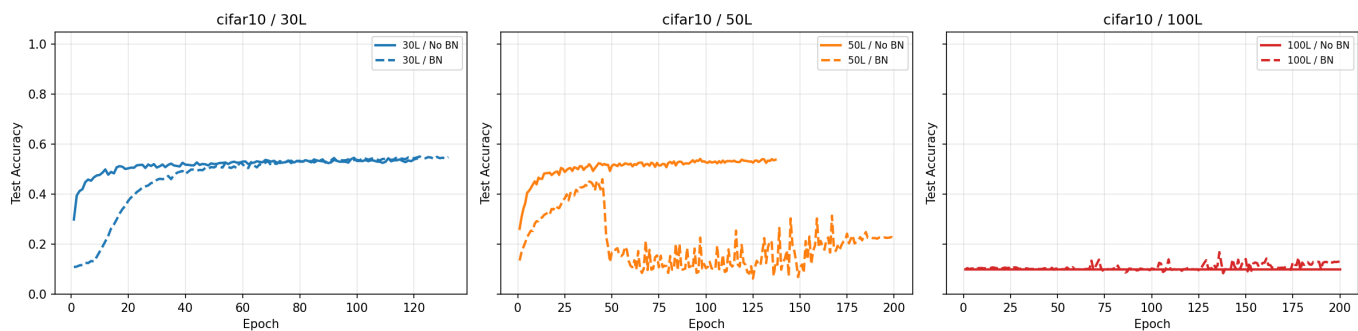


Figure 4: CIFAR-10 test accuracy by depth

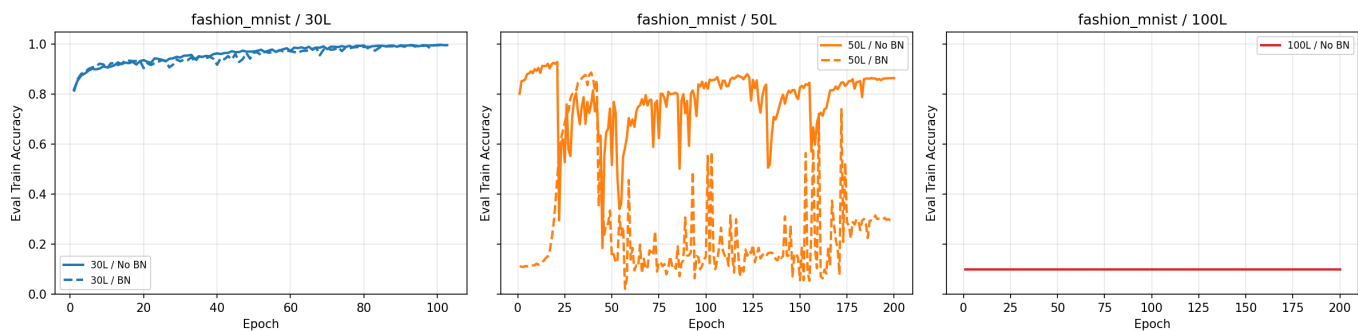


Figure 5: Fashion-MNIST eval-train accuracy by depth

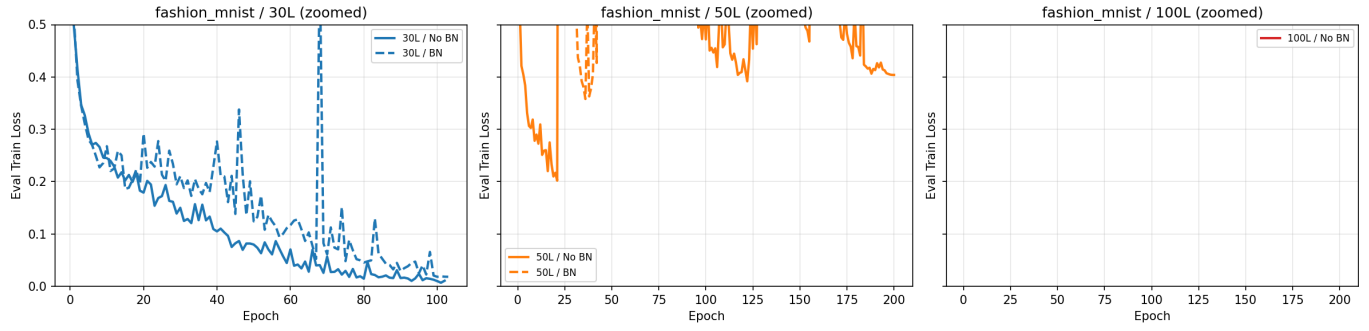


Figure 6: Fashion-MNIST eval-train loss by depth (zoomed to 0-0.5)

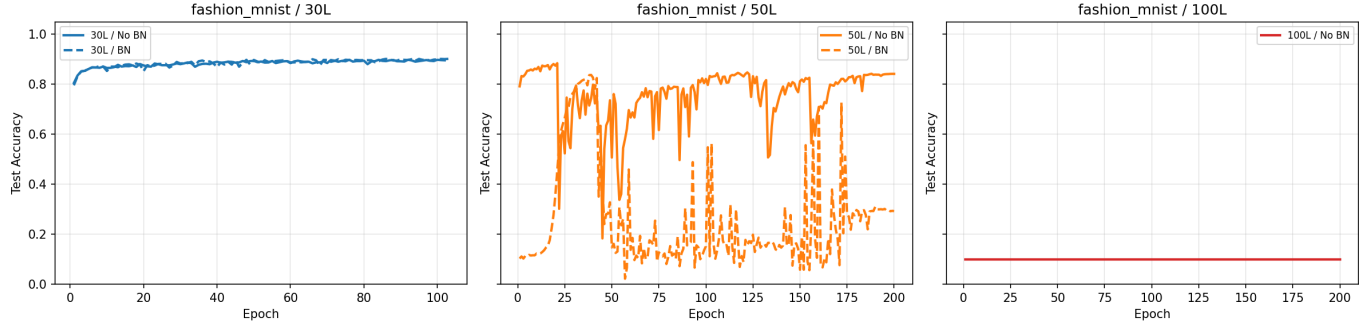


Figure 7: Fashion-MNIST test accuracy by depth

6. Sweep Coverage Summary

Architecture	Sweep Status	Tested	Sweep Best	Config Source	Result
cifar10/30L/NoBN	Complete	12/12	0.9982	Sweep winner	PASS
cifar10/30L/BN	Complete	32/32	0.9985	Sweep winner	PASS
cifar10/50L/NoBN	Incomplete (3/12)	3/12	0.4545	From 30L/NoBN winner	PASS
cifar10/50L/BN	Complete	27/32	0.8722	Sweep best	FAIL
cifar10/100L/NoBN	Not yet run	0/12	—	Best guess	FAIL
cifar10/100L/BN	Not yet run	0/32	—	From 50L/BN	FAIL
fmnist/30L/NoBN	Not yet run	0/12	—	From ci- far10/30L/NoBN	PASS
fmnist/30L/BN	Complete	32/32	0.9984	Sweep winner	PASS
fmnist/50L/NoBN	Not yet run	0/12	—	From ci- far10/50L/NoBN	FAIL
fmnist/50L/BN	Incomplete (25/32)	25/32	0.9909	Sweep near-best	FAIL
fmnist/100L/NoBN	Not yet run	0/12	—	Best guess	FAIL
fmnist/100L/BN	Not yet run	0/32	—	From 50L/BN	Timed out

Sweep status legend: - **Complete:** All configs in the grid were tested. - **Incomplete:** SLURM job timed out before finishing all configs (results saved incrementally for completed runs). - **Not yet run:** Remaining sweep jobs were submitted but results not yet downloaded. The targeted config was chosen by transferring the winning config from a related architecture.

7. Infrastructure Notes

- **Cluster:** SLURM on DLC partition (University of Haifa), single GPU per job
- **Container:** NGC PyTorch (nvidia_pt.sqsh via Pyxis/Enroot)
- **Excluded nodes:** dgx01 (always), dgx04 (old CUDA driver, version 11040)
- **Incremental saving:** JSON written after every run to prevent data loss on SLURM timeout
- **Resume support:** `--resume` flag skips completed configs when resubmitting timed-out jobs
- **Runtime per run:** ~30–70 min for 60 epochs, ~60–90 min for 200 epochs (varies by GPU node)